动态代理

1.代理模式

1.1 代理模式介绍

- 代理模式是常用的java设计模式，他的特征是代理类与委托类有同样的接口，代理类主要负责为委托类预处理消息、过滤消息、把消息转发给委托类，以及事后处理消息等。代理类与委托类之间通常会存在关联关系，一个代理类的对象与一个委托类的对象关联，代理类的对象本身并不真正实现服务，而是通过调用委托类的对象的相关方法，来提供特定的服务。简单的说就是，我们在访问实际对象时，是通过代理对象来访问的，代理模式就是在访问实际对象时引入一定程度的间接性，因为这种间接性，可以附加多种用途。
- 代理模式优点:
    - 可以隐藏委托类的实现
    - 可以实现客户与委托类间的解耦
    - 在不修改委托类代码的情况下能够做一些额外的处理

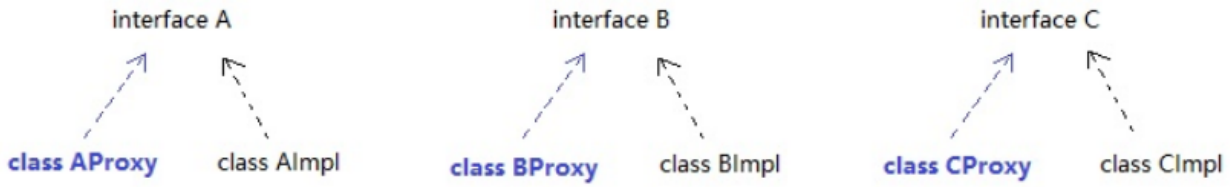
2.静态代理

2.1 静态代理介绍

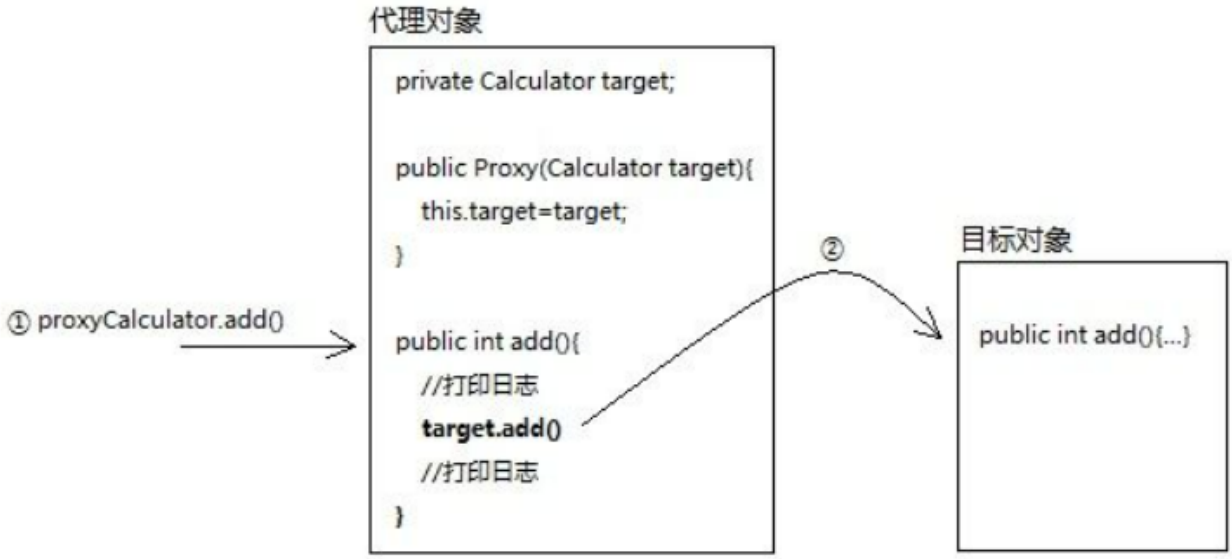
- 代理类在程序运行前就已经存在。通常情况下，静态代理中的代理类和委托类会实现同一接口或是派生自相同的父类。
- 通过代码写死目标类的代理类
- 静态代理优缺点：
    - 优点：实现原理简单
    - 缺点：程序员要手动为每一个目标类编写对应的代理类。如果当前系统已经有成百上千个类，工作量太大了。

2.2 静态代理实现

1. 为现有的每一个类都编写一个对应的代理类，并且让它实现和目标类相同的接口



2. 在创建代理对象时，通过构造器塞入一个目标对象，然后在代理对象的方法内部调用目标对象同名方法，并在调用前后写增强功能的代码。也就是说，代理对象 = 增强代码 + 目标对象（原对象）。有了代理对象后，就不用原对象了



3.动态代理

3.1 动态代理介绍

- 动态代理中代理类是运行时动态生成的(有.class文件生成，运行结束后.class文件被删除)。
- 动态代理原理：
  - 要创建一个实例，最关键的就是得到对应的Class对象，Class对象包含了一个类的所有信息，比如构造器、方法、字段等。如果我们不写代理类，这些信息从哪获取呢？代理类和目标类理应实现同一组接口(或者继承目标类)。之所以实现相同接口，是为了尽可能保证代理对象的内部结构和目标对象一致，这样我们对代理对象的操作最终都可以转移到目标对象身上，代理对象只需专注于增强代码的编写。
- 动态代理优缺点：
  - 优点：能动态生成代理类，不用每个类都手动写代理类
  - 缺点：实现原理复杂

3.2 Jdk动态代理

3.2.1 Jdk动态代理介绍

- JDK提供了java.lang.reflect.InvocationHandler接口和 java.lang.reflect.Proxy类，Jdk动态代理就是通过这两个包来实现的
- 实现Jdk动态代理目标类必须实现一个接口

3.2.2 Jdk动态代理的实现

- Jdk动态代理实现步骤：
  1. 通过实现 InvocationHandler 接口创建自己的调用处理器；
  2. 通过为 Proxy 类指定 ClassLoader 对象和一组 interface 来创建动态代理类；
  3. 通过反射机制获得动态代理类的构造函数，其唯一参数类型是调用处理器接口类型；
  4. 通过构造函数创建动态代理类实例，构造时调用处理器对象作为参数被传入。

- 代码示例：
  - 目标类实现的接口

```
package com.cxk.service;
/**
* @author cxk
* * @date 2022/2/26
*/
public interface Action {
    public void say();

    public void run();
}
```

  - 目标类

```
package com.cxk.service.impl;
import com.cxk.service.Action;
/**
* @author cxk
* @date 2022/2/26
*/
public class Man implements Action {
    public void say() {
        System.out.println("say hello");
    }

    public void run() {
        System.out.println("run run");
    }
}
```

  - 实现InvocationHandler接口

```java
package com.cxk.service.handle;
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
/**
* @author cxk
* @date 2022/2/26
*/
public class Myhandler implements InvocationHandler {
    private Object target;

    public Myhandler(Object target) {
        this.target = target;
    }

    public Object invoke(Object proxy, Method method, Object[] args)throws Throwable {
        System.out.println("action before");
        method.invoke(target,args);
        System.out.println("action after");
        return null;
    }
}
```

- 主方法

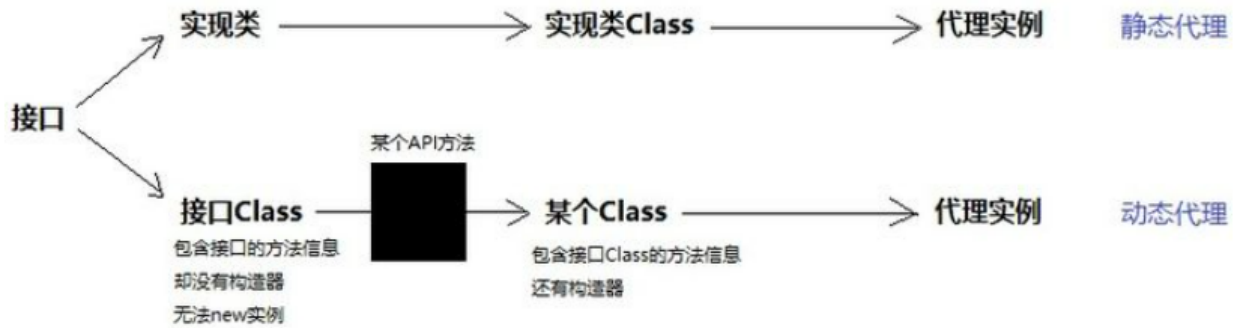
```java
import com.cxk.service.Action;
import com.cxk.service.handle.Myhandler;
import com.cxk.service.impl.Man;
import java.lang.reflect.Proxy;
/**
* @author cxk
* @date 2022/2/26
*/
public class Test {
    public static void main(String[] args) {
        Action action = new Man();

        Myhandler myhandler = new Myhandler(action);
        Action man = (Action) Proxy.newProxyInstance(
        action.getClass().getClassLoader(),
        action.getClass().getInterfaces(),
        myhandler);

        man.run();
        man.say();
    }
}
```

- 运行结果

```
D:\Develop\jdk1.8.0_201\bin\java ...
action before
run run
action after
action before
say hello
action after
```

3.2.3 Jdk动态代理实现原理

- Jdk动态代理中代理类是通过Proxy类中getProxyClass(ClassLoader, interfaces)方法拿到代理类的Class类，参数是目标类的类加载器和目标类实现的接口，通过代理类的Class类，就能生成实例。getProxyClass()这个方法，会从你传入的接口Class中，“拷贝”类结构信息到一个新的Class对象中，但新的Class对象带有构造器，是可以创建对象的。

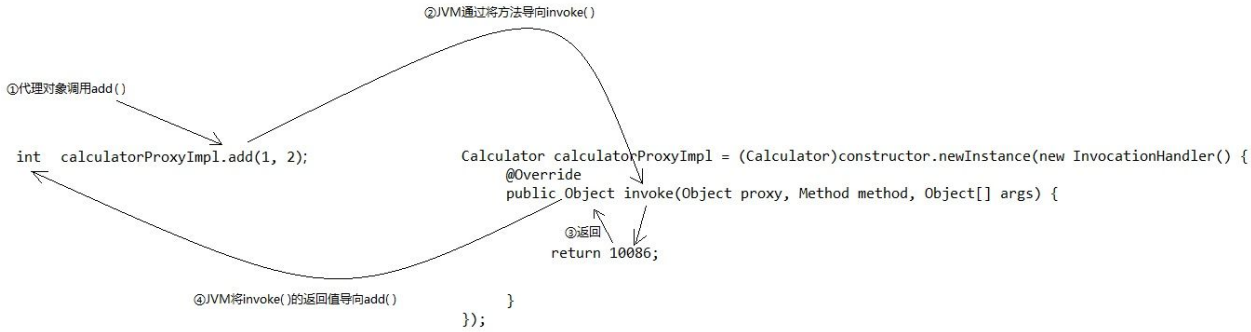


- Proxy.getProxyClass()这个方法的本质就是：以Class造Class。有了Class对象，就很好办了

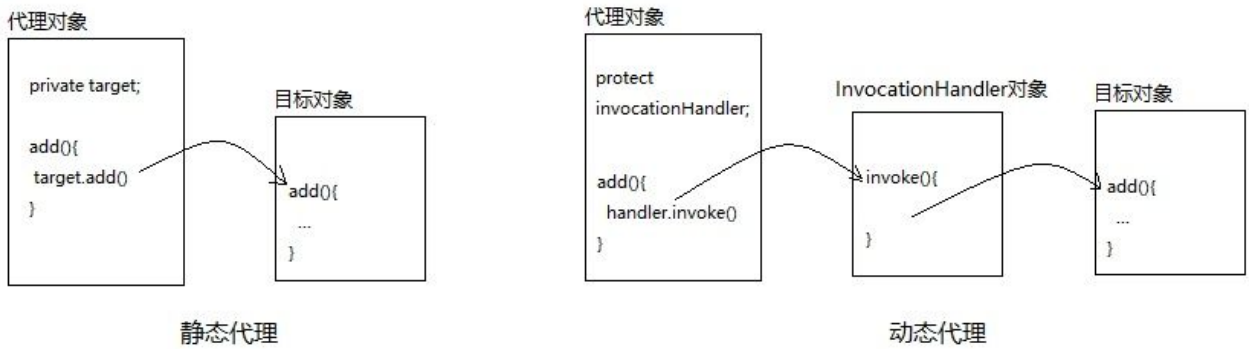
```
public class ProxyTest {
    public static void main(String[] args) throws Throwable {
        /*
         * 参数1：Calculator的类加载器（当初把Calculator加载进内存的类加载器）
         * 参数2：代理对象需要和目标对象实现相同接口Calculator
         * */
        Class calculatorProxyClazz = Proxy.getProxyClass(Calculator.class.getClassLoader(), Calculator.class);
        //得到有参构造器 $Proxy0(InvocationHandler h)
        Constructor constructor = calculatorProxyClazz.getConstructor(InvocationHandler.class);
        //反射创建代理实例
        Calculator calculatorProxyImpl = (Calculator)constructor.newInstance(new InvocationHandler() {
            @Override
            public Object invoke(Object proxy, Method method, Object[] args) {
                return null;
            }
        });
        calculatorProxyImpl.
    }
```

```
add(int a, int b)                                  int
subtract(int a, int b)                             int
equals(Object obj)                             boolean
hashCode()                                         int
toString()                                      String
getClass()                  Class<? extends Calculator>
notify()                                          void
notifyAll()                                       void
wait()                                            void
wait(long timeout)                                void
wait(long timeout, int nanos)                     void
Press Ctrl+. to choose the selected (or first) suggestion and insert a dot afterwards >>
```

- 根据代理Class的构造器创建对象时，需要传入InvocationHandler。每次调用代理对象的方法，最终都会调用InvocationHandler的invoke()方法



- 根据代理Class的构造器创建对象时，需要传入InvocationHandler。通过构造器传入一个引用，那么必然有个成员变量去接收。没错，代理对象的内部确实有个成员变量invocationHandler，而且代理对象的每个方法内部都会调用handler.invoke()！InvocationHandler对象成了代理对象和目标对象的桥梁，不像静态代理这么直接。



- 大家仔细看上图右侧的动态代理，我在invocationHandler的invoke()方法中并没有写目标对象。因为一开始invocationHandler的invoke()里确实没有目标对象，需要我们手动new。

```
public class ProxyTest {
    public static void main(String[] args) throws Throwable {
        /*
         * 参数1：Calculator的类加载器（当初把Calculator加载进内存的类加载器）
         * 参数2：代理对象需要和目标对象实现相同接口Calculator
         * */
        Class calculatorProxyClazz = Proxy.getProxyClass(Calculator.class.getClassLoader(), Calculator.class);
        //得到有参构造器 $Proxy0(InvocationHandler h)
        Constructor constructor = calculatorProxyClazz.getConstructor(InvocationHandler.class);
        //反射创建代理实例
        Calculator calculatorProxyImpl = (Calculator) constructor.newInstance(new InvocationHandler() {
            @Override
            public Object invoke(Object proxy, Method method, Object[] args) throws Exception {
                //手动new一个目标对象
                CalculatorImpl calculatorImpl = new CalculatorImpl();
                //反射执行目标对象的方法
                Object result = method.invoke(calculatorImpl, args);
                //返回目标对象执行结果
                return result;
            }
        });
        System.out.println(calculatorProxyImpl.add( a: 1,  b: 2));
        System.out.println(calculatorProxyImpl.subtract( a: 2,  b: 1));
    }
ProxyTest › printClassInfo()
ProxyTest ×
F:\develop\Java\jdk1.8.0_171\bin\java.exe ...
3
1
```

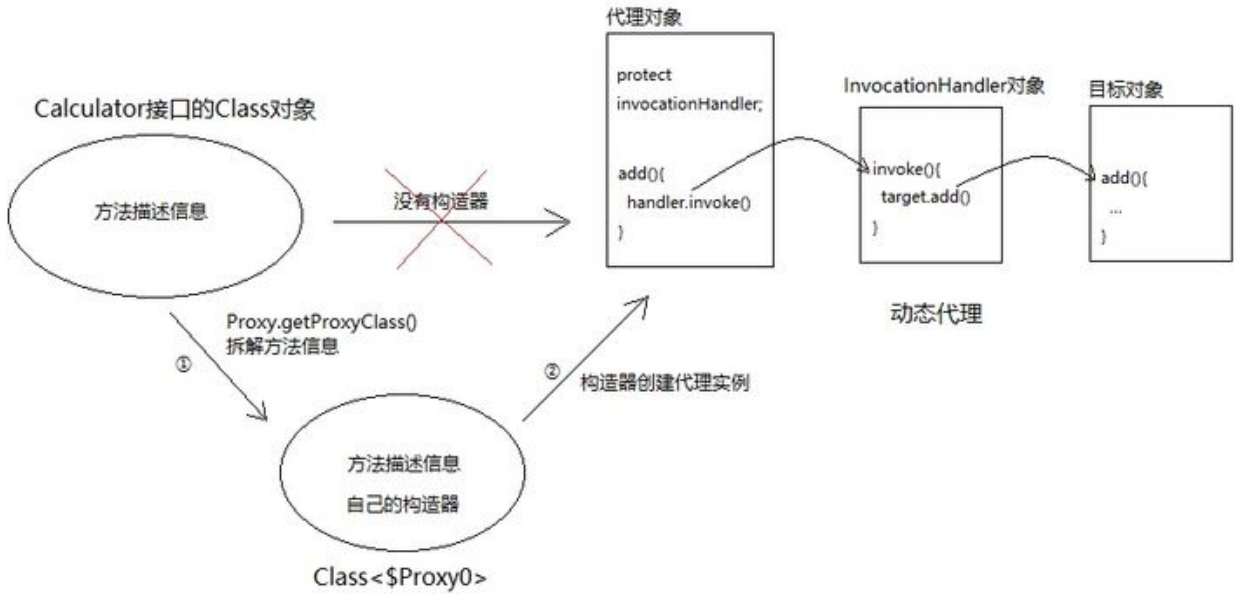
- 这种写法不够优雅，属于硬编码。我这次代理A对象，下次想代理B对象还要进来改invoke()方法，太差劲了。改进一下，让调用者把目标对象作为参数传进来

```java
public class ProxyTest {
    public static void main(String[] args) throws Throwable {
        CalculatorImpl target = new CalculatorImpl();
                //传入目标对象
                //目的：1.根据它实现的接口生成代理对象 2.代理对象调用目标对象方法
        Calculator calculatorProxy = (Calculator) getProxy(target);
        calculatorProxy.add(1, 2);
        calculatorProxy.subtract(2, 1);
    }

    private static Object getProxy(final Object target) throws Exception {
        //参数1：随便找个类加载器给它， 参数2：目标对象实现的接口，让代理对象实现相同接口
        Class proxyClazz = Proxy.getProxyClass(target.getClass().getClassLoader(),
target.getClass().getInterfaces());
        Constructor constructor = proxyClazz.getConstructor(InvocationHandler.class);
        Object proxy = constructor.newInstance(new InvocationHandler() {
            @Override
            public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
                System.out.println(method.getName() + "方法开始执行...");
                Object result = method.invoke(target, args);
                System.out.println(result);
                System.out.println(method.getName() + "方法执行结束...");
                return result;
            }
        });
        return proxy;
    }
}
```

- 这样就非常灵活，非常优雅了。无论现在系统有多少类，只要你把实例传进来，getProxy()都能给你返回对应的代理对象。就这样，我们完美地跳过了代理类，直接创建了代理对象！
- 不过实际编程中，一般不用getProxyClass()，而是使用Proxy类的另一个静态方法：Proxy.newProxyInstance()，直接返回代理实例，连中间得到代理Class对象的过程都帮你隐藏

```java
public class ProxyTest {
    public static void main(String[] args) throws Throwable {
        CalculatorImpl target = new CalculatorImpl();
        Calculator calculatorProxy = (Calculator) getProxy(target);
        calculatorProxy.add(1, 2);
        calculatorProxy.subtract(2, 1);
    }

    private static Object getProxy(final Object target) throws Exception {
        Object proxy = Proxy.newProxyInstance(
                target.getClass().getClassLoader(),/*类加载器*/
                target.getClass().getInterfaces(),/*让代理对象和目标对象实现相同接口*/
                new InvocationHandler(){/*代理对象的方法最终都会被JVM导向它的invoke方法*/
                    public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
                        System.out.println(method.getName() + "方法开始执行...");
                        Object result = method.invoke(target, args);
                        System.out.println(result);
                        System.out.println(method.getName() + "方法执行结束...");
                        return result;
                    }
                }
        );
        return proxy;
    }
}
```

- newProxyInstance 方法里面获取到代理类，如果类的作用不能访问，使其能被访问到，最后实例化代理类。这段代码中最为核心的是获取代理类的getProxyClass0方法。
- 如果 proxyClassCache 缓存中存在指定的代理类，则从缓存直接获取；如果不存在，则通过 ProxyClassFactory 创建代理类。 至于为什么接口最大为 65535，这个是由字节码文件结构和 Java 虚拟机规定的，具体可以通过研究字节码文件了解。
- 总结 ： 动态代理就是要动态生成代理类，而生成类就需要Class文件，Jdk动态代理通过反射从目标类实现的接口中拿到Class结构，动态生成代理类。实现InvocationHandler接口重写invoke方法进行功能增强，在代理类调用方法时候，会通过反射拿到目标Method，执行目标类中的方法和自己写的增强代码



- 链接：https://www.zhihu.com/question/20794107/answer/658139129

3.3 Cglib动态代理

3.3.1 Cglib动态代理介绍

- JDK实现动态代理需要实现类通过接口定义业务方法，对于没有接口的类，如何实现动态代理呢，这就需要CGLib了。CGLib采用了非常底层的字节码技术，其原理是通过字节码技术为一个类创建子类，并在子类中采用方法拦截的技术拦截所有父类方法的调用，顺势织入横切逻辑。JDK动态代理与CGLib动态代理均是实现Spring AOP的基础。
- Cglib是一个强大的高性能的代码生成包。

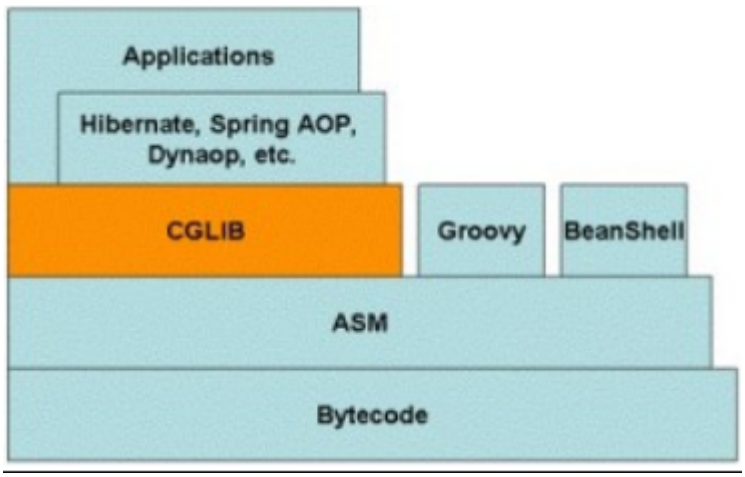
```
1. 它广泛的被许多AOP的框架使用，例如：Spring AOP和dynaop，为他们提供方法的interception（拦截）；
```

```
2. hibernate使用CGLIB来代理单端single-ended(多对一和一对一)关联（对集合的延迟抓取，是采用其他机制实现的
```

```
3. EasyMock和jMock是通过使用模仿（moke）对象来测试java代码的包。
```

```
4. 它们都通过使用CGLIB来为那些没有接口的类创建模仿（moke）对象。
```

- Cglib底层



  - 最底层的是字节码ByteCode，字节码是Java为了保证“一次编译，到处运行”而产生的一种虚拟的指令格式。
  - 位于字节码之上的是ASM，这是一种直接操作字节码的框架，应用ASM需要对Java字节码、Class结构比较熟悉。
  - 位于ASM之上的是CGLIB、Groovy、BeanShell，后两种并不是Java体系中的内容而是脚本语言，它们通过ASM框架生成字节码变相执行Java代码，这说明在JVM中执行程序并不一定非要写Java代码，只要你能生成Java字节码，JVM并不关心字节码的来源，当然通过Java代码生成的JVM字节码是通过编译器直接生成的，算是最“正统”的JVM字节码。
  - 位于CGLIB、Groovy、BeanShell之上的就是Hibernate、Spring AOP这些框架了。
  - 最上层的是Applications，即具体应用，一般都是一个Web项目或者本地跑一个程序。
  - 链接：https://www.jianshu.com/p/a20d5c949ac4

3.3.2 Cglib动态代理实现

- 代码实例
  - 目标类

```
public class HelloService {
  public HelloService() {
      System.out.println("HelloService Constructor...");
  }

  /**
   * 该方法不能被子类覆盖，Cglib是无法代理final修饰的方法的
   */
  final public String sayOthers(String name) {
      System.out.println("HelloService...sayOthers..." + name);
      return null;
  }

  public void sayHello() {
      System.out.println("HelloService...sayHello...");
  }
}
```

  - MethodInterceptor类

```
public class MyMethodInterceptor implements MethodInterceptor {
  /**
   * cglib
   *
   * @param o cglib生成的代理对象
   * @param method 被代理对象的方法
   * @param objects 方法入参
   * @param methodProxy 代理方法
   * @return
   * @throws Throwable
   */
  @Override
  public Object intercept(Object o, Method method, Object[] objects, MethodProxy
methodProxy) throws Throwable {
      System.out.println("before advice...");
      Object object = methodProxy.invokeSuper(o, objects);
      System.out.println("after advice...");
      return object;
  }
}
```

  - 测试类

```java
public class Main {
  public static void main(String[] args) {
      //代理类class文件存入本地磁盘方便我们反编译查看源码
      System.setProperty(DebuggingClassWriter.DEBUG_LOCATION_PROPERTY,
"F:\\workspace\\demo");
      //通过CGLIB动态代理获取代理对象的过程
      Enhancer enhancer = new Enhancer();
      //设置enhancer对象的父类
      enhancer.setSuperclass(HelloService.class);
      //设置enhancer的回调对象
      enhancer.setCallback(new MyMethodInterceptor());
      //创建代理对象
      HelloService proxy = (HelloService) enhancer.create();
      //通过代理对象调用目标方法
      proxy.sayHello();
  }
}
```

- 执行结果

```
HelloService Constructor...
before advice...
HelloService...sayHello...
after advice...
```

3.3.3 Cglib动态代理实现原理

- 我们先从生成的字节码入手，看看cglib如何利用它们来完成代理的。HelloService$EnhancerByCGLIB$750c1a60.class。前面说了，cglib是采用继承的方式，所以结果也不难看出，生成的字节码类继承了真实类，这样是可以重写其父类的方法，并且还可以添加一些额外的逻辑。

```
public class HelloService$$EnhancerByCGLIB$$750c1a60 extends HelloService implements Factory {
    private boolean CGLIB$BOUND;
    public static Object CGLIB$FACTORY_DATA;
    private static final ThreadLocal CGLIB$THREAD_CALLBACKS;
    private static final Callback[] CGLIB$STATIC_CALLBACKS;
    private MethodInterceptor CGLIB$CALLBACK_0;
    private static Object CGLIB$CALLBACK_FILTER;
    //真实对象的方法
    private static final Method CGLIB$sayHello$0$Method;
    //代理对象的代理方法（都与真实对象的方法一一对应）
    private static final MethodProxy CGLIB$sayHello$0$Proxy;
    private static final Object[] CGLIB$emptyArgs;
    private static final Method CGLIB$equals$1$Method;
    private static final MethodProxy CGLIB$equals$1$Proxy;
    private static final Method CGLIB$toString$2$Method;
    private static final MethodProxy CGLIB$toString$2$Proxy;
    private static final Method CGLIB$hashCode$3$Method;
    private static final MethodProxy CGLIB$hashCode$3$Proxy;
    private static final Method CGLIB$clone$4$Method;
    private static final MethodProxy CGLIB$clone$4$Proxy;

    //静态代码块，会优先加载
    static void CGLIB$STATICHOOK1() {
        CGLIB$THREAD_CALLBACKS = new ThreadLocal();
        CGLIB$emptyArgs = new Object[0];
        //利用Class.forName创建对象，后面的全限定名就是代理对象
        Class var0 =
Class.forName("com.alibaba.proxy.cglib.HelloService$$EnhancerByCGLIB$$750c1a60");
        //这里的var1会在下面进行创建，这里只是声明
        Class var1;
        Method[] var10000 = ReflectUtils.findMethods(new String[]{"equals", "
(Ljava/lang/Object;)Z", "toString", "()Ljava/lang/String;", "hashCode", "()I", "clone", "
()Ljava/lang/Object;"}, (var1 = Class.forName("java.lang.Object")).getDeclaredMethods());
        CGLIB$equals$1$Method = var10000[0];
        CGLIB$equals$1$Proxy = MethodProxy.create(var1, var0, "(Ljava/lang/Object;)Z", "equals",
"CGLIB$equals$1");
        CGLIB$toString$2$Method = var10000[1];
        CGLIB$toString$2$Proxy = MethodProxy.create(var1, var0, "()Ljava/lang/String;",
"toString", "CGLIB$toString$2");
        CGLIB$hashCode$3$Method = var10000[2];
        CGLIB$hashCode$3$Proxy = MethodProxy.create(var1, var0, "()I", "hashCode",
"CGLIB$hashCode$3");
        CGLIB$clone$4$Method = var10000[3];
        CGLIB$clone$4$Proxy = MethodProxy.create(var1, var0, "()Ljava/lang/Object;", "clone",
"CGLIB$clone$4");
        //上面的var1声明，这里进行创建对象并赋值。同样是通过Class.forName创建，后面是真实对象的全限定名
        CGLIB$sayHello$0$Method = ReflectUtils.findMethods(new String[]{"sayHello", "()V"}, (var1
= Class.forName("com.alibaba.proxy.cglib.HelloService")).getDeclaredMethods())[0];
        //创建一个MethodProxy对象，将代理类、真实类、代理方法、真实方法都作为入参
        CGLIB$sayHello$0$Proxy = MethodProxy.create(var1, var0, "()V", "sayHello",
"CGLIB$sayHello$0");
    }
```

- CGLIB$sayHello$0()方法，是代理方法，methodProxy.invokeSuper会调用。下面的sayHello是真实方法，methodProxy.invoke会调用。这里解释了为什么在拦截器中调用methodProxy.invoke会死循环，因为下面的代码中一直在调用intercept。

```
final void CGLIB$sayHello$0() {
    super.sayHello();
}

public final void sayHello() {
    MethodInterceptor var10000 = this.CGLIB$CALLBACK_0;
    if (var10000 == null) {
        CGLIB$BIND_CALLBACKS(this);
        var10000 = this.CGLIB$CALLBACK_0;
    }

    if (var10000 != null) {
        //调用intercept方法，也就是在自定义的Intercepter类中实现的方法。
        var10000.intercept(this, CGLIB$sayHello$0$Method, CGLIB$emptyArgs,
CGLIB$sayHello$0$Proxy);
    } else {
        super.sayHello();
    }
}
```

- 一直到上面，简单说一下流程：代理对象调用sayHello，然后调用拦截器intercept，然后执行拦截器中的前面代码，然后执行invokeSuper，会执行CGLIB$sayHello$0方法，然后调用父类sayHello，然后执行拦截器后面代码。
- 那么从invokeSuper方法中为什么就能调用到CGLIB$sayHello$0()方法呢？
  - MethodProxy.create：在代理类中create一个MethodProxy，将被代理对象与方法传入Signature1，将代理对象与方法传入Signature2，将代理类对象与被代理类对象传入MethodProxy.CreateInfo，Signature1与MethodProxy.CreateInfo可以认为是实体类存储信息的。
  - 调用methodProxy.invokeSuper了，调用invokeSuper时又会首先调用init方法。
  - init方法就是给在create方法中没有赋值的fastclassInfo赋值，将createInfo与Signature1与Signature2中的信息取出，转成fastclass，如果fastclass在缓存中有就从缓存中取，没有的话就生成新的fastclass。
  - 通过Signature1与Signature2获得方法的索引，存入i1与i2。
  - 通过fci.f2.invoke(fci.i2, obj, args)，fastclass反射调用代理对象的代理方法的索引，直接定位到代理方法。
  - 代理方法中调用super.sayHello()

```java
public class MethodProxy {

    private Signature sig1;

    private Signature sig2;

    private CreateInfo createInfo;

    private final Object initLock = new Object();

    private volatile FastClassInfo fastClassInfo;

    /**
     * For internal use by {@link Enhancer} only; see the {@link
org.springframework.cglib.reflect.FastMethod} class
     * for similar functionality.
     */
    public static MethodProxy create(Class c1, Class c2, String desc, String name1, String name2)
{
        MethodProxy proxy = new MethodProxy();
        proxy.sig1 = new Signature(name1, desc);
        proxy.sig2 = new Signature(name2, desc);
        proxy.createInfo = new CreateInfo(c1, c2);
        return proxy;
    }
```

```java
public Object invokeSuper(Object obj, Object[] args) throws Throwable {
    try {
        init();
        FastClassInfo fci = fastClassInfo;
        //fastCLass.invoke(代理方法索引，代理对象，方法参数)
        return fci.f2.invoke(fci.i2, obj, args);
    }
    catch (InvocationTargetException e) {
        throw e.getTargetException();
    }
}
```

- MethodProxy调用流程
  - 首先main方法中调用getInstance调用Enhancer创建代理类class文件，并将代理类引用返回。
  - 然后调用代理对象的sayHello()
  - 然后通过反编译代理类，查看sayHello方法，发现在sayHello中会调用拦截器方法。
  - 然后在拦截器方法中会通过MethodProxy调用invokeSuper方法。
- FastClass机制
  - Cglib动态代理执行代理方法效率之所以比JDK高是因为Cglib采用了FastClass机制，它为代理类和被代理类各生成了一个class，这个class会为代理类与被代理类的方法分类index。这个index作为方法参数，FastClass可以直接定位到要调用的方法进行调用，这样省去了反射调用，所以效率比JDK动态代理快。FastClass不是与代理类一起生成的，而是在第一次执行MethodProxy invoke/invokeSuper时生成的并放入缓存。

```java
public class HelloService$$FastClassByCGLIB$$8656ab6f extends FastClass {
    public HelloService$$FastClassByCGLIB$$8656ab6f(Class var1) {
        super(var1);
    }

    public int getIndex(Signature var1) {
        String var10000 = var1.toString();
        switch(var10000.hashCode()) {
        case -1488716497:
            if (var10000.equals("sayOthers(Ljava/lang/String;)Ljava/lang/String;")) {
                return 1;
            }
            break;
        }

        return -1;
    }

    public Object invoke(int var1, Object var2, Object[] var3) throws InvocationTargetException {
        HelloService var10000 = (HelloService)var2;
        int var10001 = var1;

        try {
            switch(var10001) {
            case 0:
                var10000.sayHello();
                return null;
            case 1:
                return var10000.sayOthers((String)var3[0]);
            case 2:
                return new Boolean(var10000.equals(var3[0]));
            case 3:
                return var10000.toString();
            case 4:
                return new Integer(var10000.hashCode());
            }
        } catch (Throwable var4) {
            throw new InvocationTargetException(var4);
        }
    }
}
```

- 因为生成的FastClass代理类较长，这里只截取其中一部分，只为了说明问题。这里面包含了getIndex方法和invoke方法。getIndex方法会为代理类和真实类的方法分类index，这样每个index会对应到具体方法，那么在上面那个invokeSuper那里，实际上就是用fastClass对象调用的invoke方法，传入的参数就包含这个index，那么在invoke方法中，就直接根据index定位到具体方法执行了。
- 链接：https://www.jianshu.com/p/a20d5c949ac4

3.4 Jdk动态代理与Cglib动态代理比较

3.4.1 两者区别

1. jdk动态代理利用反射机制生成一个实现代理接口的匿名类，在调用具体方法前调用InvokeHandler来处理。cglib动态代理是利用asm开源包，对代理对象类的class文件加载进来，通过修改其字节码生成子类来处理。
2. jdk和cglib都是在运行期生成字节码，jdk是直接写class字节码，cglib使用ASM框架写class字节码，cglib代理实现更复杂，生成代理类比jdk效率低。
3. jdk调用代理方法，是通过反射机制调用，cglib是通过FastClass机制直接调用方法，cglib执行效率更高

3.4.2 Spring如何选择调用两种

- 当Bean实现接口时，Spring就会用JDK的动态代理
- 当Bean没有实现接口时，Spring使用CGLib来实现
- 可以强制使用CGLib（在Spring配置中加入）

3.4.3 两者性能比较

- 使用CGLib实现动态代理，CGLib底层采用ASM字节码生成框架，使用字节码技术生成代理类，在JDK1.6之前比使用Java反射效率要高。唯一需要注意的是，CGLib不能对声明为final的方法进行代理，因为CGLib原理是动态生成被代理类的子类。
- 在JDK1.6、JDK1.7、JDK1.8逐步对JDK动态代理优化之后，在调用次数较少的情况下，JDK代理效率高于CGLib代理效率，只有当进行大量调用的时候，JDK1.6和JDK1.7比CGLib代理效率低一点，但是到JDK1.8的时候，JDK代理效率高于CGLib代理